

TÀI LIỆU WORKSHOP: LÀM CHỦ SOLID VÀ DESIGN PATTERNS

I. THÔNG TIN CHUNG (METADATA)

- **Tiêu đề workshop:** Nhập môn SOLID và Design Pattern trong Kỹ thuật Phần mềm
 - **Mô tả dự kiến:** Một buổi workshop chuyên sâu về tư duy thiết kế hệ thống dành cho sinh viên IT.
 - **Giá của workshop:** 0 VNĐ (Miễn phí)
 - **Địa điểm của workshop:** Phòng máy 1, Tòa nhà I, Trường ĐH Khoa học Tự nhiên, 227 Nguyễn Văn Cừ, Quận 5.
 - **Tên người hướng dẫn:** Dương Gia Huy
 - **Số chỗ tối đa của workshop:** 50
 - **Thời gian bắt đầu:** 08:30 - 15/05/2026
 - **Thời gian kết thúc:** 11:30 - 15/05/2026
-

II. NỘI DUNG CHI TIẾT (AGENDA)

Phần 1: Tại sao code của chúng ta thường trở thành “đồng rác”?

- Phân tích các dấu hiệu của một mã nguồn kém: Rigidity (Cứng nhắc), Fragility (Dễ vỡ), và Immobility (Khó tái sử dụng).
- Giới thiệu triết lý Clean Code và tầm quan trọng của việc tách biệt mối quan tâm (Separation of Concerns).

Phần 2: Giải mã 5 nguyên lý SOLID

- **Single Responsibility (SRP):** Tại sao một Class chỉ nên có một lý do để thay đổi? Ví dụ về việc tách Logic xử lý dữ liệu và Logic hiển thị UI.
- **Open/Closed (OCP):** Kỹ thuật mở rộng tính năng mà không cần sửa đổi mã nguồn cũ.
- **Liskov Substitution (LSP):** Đảm bảo tính đúng đắn dẫn khi sử dụng đa hình.
- **Interface Segregation (ISP):** Tránh việc ép buộc các class phụ thuộc vào các phương thức chúng không sử dụng.
- **Dependency Inversion (DIP):** Cách sử dụng Dependency Injection để giảm sự phụ thuộc cứng (Hard-coded).

Phần 3: Ứng dụng Design Patterns phổ biến

- **Strategy Pattern:** Thay đổi thuật toán linh hoạt trong runtime (Ví dụ: Các loại hình thanh toán khác nhau).

- **Observer Pattern:** Xây dựng hệ thống thông báo/sự kiện (Ví dụ: Hệ thống thông báo trong game).
- **Factory Method:** Quản lý việc khởi tạo đối tượng phức tạp.

Phần 4: Thực hành và Q&A

- Refactor một đoạn code “xấu” sang code áp dụng SOLID.
- Giải đáp thắc mắc về lộ trình trở thành Software Architect.

Tài liệu nội bộ dự án Unihub.